

Aula 6 - Complexidade Computacional

Problema de ordenação

Imaginem o seguinte problema: temos uma lista de números, e queremos ordená-la em ordem crescente. As únicas operações permitidas são:

1. Fazer comparações entre dois números
2. Trocar dois números

Seja a seguinte lista: **6 8 2 3 1 4 5 7** como podemos ordenar ela?

Ideia 1

Passamos pelo vetor inteiro, achamos o menor elemento, colocamos na primeira posição. Fazemos isso de novo, olhando da segunda posição em diante, procuramos o menor elemento e colocamos na segunda posição.

Repetimos isso até que o vetor esteja ordenado.

Ideia 2

Passamos pelo vetor inteiro, para cada dois números adjacentes x e y , colocamos o menor antes do maior e andamos para frente no vetor. Repetimos isso até que os números maiores estejam mais para frente no vetor do que os menores, ordenando-o.

1. As duas ideias funcionam?
2. Qual delas é melhor?

Resposta: ambas funcionam! E ambas são igualmente ruins!

Como comparar algoritmos?

Digamos que eu (Matias) fui o autor da ideia 1. Enquanto o Bento foi o autor da ideia 2. Como eu consigo dizer para o Bento "meu algoritmo é objetivamente melhor que o seu"?

Uma ideia seria rodar os dois e ver qual roda mais rápido. Mas isso abre margem para inúmeras interferências externas (velocidade de processamento do computador, condições internas do computador no momento da execução...). Então tempo não é uma boa ideia.

A resposta é: número de operações. O número de operações realizadas por um algoritmo é constante independente de onde ele é rodado. Isso possibilita comparar bem o meu algoritmo com o do Bento.

Notação big O

Vamos olhar para a ideia 1. Qual o pior caso para ela? É o caso em que ninguém está no lugar certo. Um "pior caso" seria o vetor ordenado de forma oposta: **8 7 6 5 4 3 2 1**.

Para ordenar o primeiro elemento, precisaríamos fazer 8 comparações. Para ordenar o segundo, precisaríamos fazer 7 comparações. O terceiro, 6. E assim por diante...

Ou seja:

$$\sum_{i=1}^8 i \text{ operações}$$

E se fosse um "pior caso" com n elementos?

Teríamos:

$$\sum_{i=1}^n i = 1 + 2 + 3 + \dots + n = \frac{(n+1)n}{2} = \frac{n^2 + n}{2}$$

Agora, imaginemos que $n = 10^2$. Temos que $n^2 = 10^4$.

Já se $n = 10^9$ temos $n^2 = 10^{18}$. Concordamos que 10^{18} é MUITO maior do que 10^9 . De forma que esse 10^9 é desprezível perto do 10^{18} . Bem como a fração sobre 2.

Então, podemos dizer que, para n extremamente grande, no pior caso, meu algoritmo faz n^2 operações.

Isso é a ideia da notação big O. Temos uma função $f(n)$ que descreve o crescimento do número de operações do meu algoritmo em função do tamanho da entrada n . A gente ignora todos os termos que estão em função de n (exceto o de maior grau) e ignora também todas as constantes. Assim, podemos dizer que o meu algoritmo é $O(n^2)$.

Deem uma olhada nos seguintes exemplos de notação big O:

- $n^3 + n^2 + n + 1 \rightarrow O(n^3)$
- $n^4 + 9999999 + n/2 \rightarrow O(n^4)$
- $\log_2 n + 2 \rightarrow O(\log n)$

Vou me poupar de analisar o algoritmo da ideia 2, mas, no pior caso, o algoritmo da ideia 2 também é $O(n^2)$. Portanto, ambos são iguais em termos de eficiência.

Por curiosidade, segue a comparação das ordens de complexidade possíveis:

$$O(1) < O(\log n) < O(\sqrt{n}) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$$